



斯坦福 CS336

从零开始的语言建模 | 2026 年春季

p02: PyTorch (einops)

讲座笔记

视频来源: Bilibili BV12gLx6xEcq p02

整理日期: June 8, 2026

Abstract

本笔记整理自 Stanford CS336 “Language Modeling from Scratch” (Spring 2026) 第二讲。本讲核心围绕“资源核算”展开，从底层张量存储与精度类型，到中层 einops 的张量操作抽象，再到上层训练全流程的计算与内存分析。主要内容包括：PyTorch 张量基础与数值精度 (FP32/FP16/BF16/FP8/FP4)、einops 命名维度操作 (einsum / reduce / rearrange)、FLOPs 核算与 MFU (模型浮点利用率)、算术强度与 Roofline 模型、以及训练前向/反向/优化器的内存与计算分析。

Contents

1	课程回顾与张量基础	4
1.1	课程回顾与项目结果	4
1.2	资源核算的目标与意义	4
1.3	张量：存储一切的基本单元	5
1.4	数值精度类型与内存占用	5
1.4.1	FP32：基准精度	5
1.4.2	FP16：半精度及其局限	5
1.4.3	BF16：动态范围与精度的折中	6
1.4.4	FP8 与 FP4：极限量化	6
1.5	混合精度训练	6
1.6	GPU 内存搬运	7
2	einops：命名维度与爱因斯坦求和	7
2.1	设计动机：索引 vs. 命名维度	8
2.2	einsum：广义矩阵乘法	8
2.2.1	矩阵乘法	8
2.2.2	批量矩阵乘法	9
2.2.3	省略号：批处理维度的泛化	9
2.3	reduce：广义规约操作	9
2.4	rearrange：维度的拆分与合并	10
2.4.1	注意力头示例：拆分隐藏维度	10
2.4.2	行主序约定	11
2.5	小结	11
3	计算成本：FLOPs、硬件峰值与 MFU	11
3.1	FLOPs 与 FLOPS 的辨析	11
3.2	NVIDIA H100 规格表解读	12
3.3	线性层 FLOPs 的推导	12
3.4	逐元素操作的 FLOPs	13
3.5	GPU 计时方法与 CUDA 同步	13
3.5.1	为什么需要 <code>torch.cuda.synchronize()</code>	13
3.5.2	多次运行取平均	14
3.5.3	从计时结果计算实际 FLOPS	14
3.6	MFU：模型浮点利用率	14
3.6.1	定义	14
3.6.2	典型值与工程含义	14
3.6.3	为什么 MFU 通常只有 0.5?	15
3.7	本章小结	15
4	算术强度与屋顶线模型	15
4.1	硬件简图：计算与通信的双重瓶颈	15
4.1.1	HBM 与计算核心	15
4.1.2	计算与通信的重叠	16
4.2	算术强度的定义	16
4.2.1	操作强度与加速器强度	16
4.2.2	内存受限与计算受限的判别	17
4.3	典型操作的算术强度对比	17

4.3.1	ReLU 激活函数: $I \approx 0.25$	17
4.3.2	GELU 激活函数: $I \approx 5$	17
4.3.3	点积 (Dot Product): $I \approx 0.5$	18
4.3.4	矩阵-向量乘法 (Mat-Vec): 内存受限	18
4.3.5	矩阵-矩阵乘法 (Mat-Mat): $I \approx N/3$, 计算受限	18
4.4	Transformer 的算术强度: 训练 vs 推理	19
4.4.1	训练阶段: 矩阵乘为主, 计算受限	19
4.4.2	推理阶段: 矩阵-向量为, 内存受限	19
4.5	屋顶线模型 (Roofline Model)	19
4.5.1	模型描述	19
4.5.2	各操作在 Roofline 上的位置	20
4.6	MFU 偏低与内存瓶颈的关系	20
4.7	本章小结	21
5	训练全流程: 前向、反向、优化器与内存占用	21
5.1	深度网络前向传播	21
5.2	从标量到矩阵: 反向传播的直觉	22
5.3	反向传播与链式法则的 einops 表示	22
5.4	梯度计算量 = $2 \times$ 前向	23
5.5	总训练 FLOPs = $6 \cdot N \cdot D$ 的推导	23
5.6	优化器状态内存: Adagrad 与 Adam	24
5.6.1	Adagrad	24
5.6.2	Adam	24
5.7	训练内存占用核算	24
5.8	梯度累积	26
5.9	激活检查点	26
5.9.1	原理与内存节省	26
5.9.2	极端情况: L^2 重计算	27
5.9.3	最优平衡点: $\sqrt{L}$ 个检查点	27
5.10	本章小结	27
	总结与展望	29

1 课程回顾与张量基础

1.1 课程回顾与项目结果

本课程的核心目标是：在有限的算力、内存（有时还包括数据）条件下，训练出性能最优的模型。讲师指出，在本次课程中，数据并非限制因素，因此我们的重点是最大化训练的计算效率。

在回顾上一讲内容时，讲师分享了班级项目的运行结果。该项目旨在通过大量小模型的实验验证缩放定律（Scaling Laws）：

- 全班同学的项目累计消耗了约 10^{23} 次浮点运算；
- 每条实验曲线本质上是一条等浮点运算曲线（iso-FLOP curve），由众多小模型的运行结果拟合而成；
- 最终拟合出的缩放定律成功预测了最优计算资源配置点对应的损失值；
- 实际运行得到的损失与预测值相差不到 0.05，验证了缩放定律的惊人准确性。

讲师点评

讲师在此特别强调：“每写一行代码都考虑一下性能特性。”资源核算（resource accounting）是系统层面训练大模型的基本功，希望同学们养成习惯。

1.2 资源核算的目标与意义

在优化计算效率之前，我们必须先了解特定计算任务的效率特性，即掌握其计算特性与内存特性。学完本课后，应当能够回答如下典型问题：

例 1.1. 在 1024 块 H100 GPU 上，训练一个 700 亿参数的模型，处理 15 万亿个 token，需要多长时间？

估算方法如下：

1. 总浮点运算次数： $\text{FLOPs} = 6 \cdot N \cdot D$ ，其中 N 为参数量， D 为 token 数量；
2. 查阅 H100 规格表，获取其 BF16 峰值性能；
3. 引入模型浮点利用率（Model FLOPs Utilization, MFU），典型值约为 0.5；
4. 估算硬件每天可提供的浮点运算次数，进而求得天数。

按上述方法估算，答案约为 **143** 天。

例 1.2. 使用 **8-bit** 优化器状态量化（如 bitsandbytes¹），在 8 块 H100（每块 80 GB HBM）上能训练的最大模型是多大？

估算思路：每块 H100 有 80 GB 高带宽内存（HBM），每个参数约需 $2 + 2 + 4 = 8$ 字节（BF16 参数 + BF16 梯度 + FP32 优化器状态），使用 8-bit 优化器后优化器状态可压缩至约 1 字节，粗略估算可支持约 **530** 亿参数（未计入激活值）。激活值内存取决于批次大小和序列长度，因此这只是一个粗略估计。

重点不在于精确计算每一项细节，而是对整个训练过程的资源消耗有数量级的把握。

¹8-bit 优化器量化技术将优化器状态从 FP32 压缩到 8-bit，使每参数内存从 $2 + 2 + 4 = 8$ 字节降至约 $2 + 2 + 1 = 5$ 字节。

1.3 张量：存储一切的基本单元

讲师指出，我们自下而上逐步构建理解，最底层是张量（**Tensor**）。张量是存储一切的基本单元——无论是参数、梯度、优化器状态、数据还是激活值，本质上都是张量。

以 DeepSeek-V3 模型为例，模型本身由许多不同的张量组成，每个张量都有特定的形状（`shape`）与精度（`dtype`）。张量涵盖了向量和矩阵，并且可以推广到任意数量的维度。

1.4 数值精度类型与内存占用

张量的存储空间取决于其数据类型。对于浮点数张量，常见的精度类型及其内存占用如表 1 所示。

Table 1: 常见浮点精度类型对比

类型	总位数	符号位	指数位	尾数位	每元素字节
FP32 (单精度)	32	1	8	23	4
FP16 (半精度)	16	1	5	10	2
BF16 (Brain Float 16)	16	1	8	7	2
FP8 (E4M3)	8	1	4	3	1
FP8 (E5M2)	8	1	5	2	1
FP4 / MVFP4	4	—	—	—	0.5

1.4.1 FP32：基准精度

FP32（32-bit 浮点数，又称单精度）由 1 个符号位、8 个指数位和 23 个尾数位组成。指数位决定了动态范围，尾数位提供了数值精度。在科学计算传统中，FP32 是浮点数的默认基准；若需更高精度，则使用 FP64（双精度）。

在 PyTorch 中，默认张量类型即为 FP32：

```
1 import torch
2
3 # 默认创建 FP32 张量
4 x = torch.randn(4, 8)           # 形状 (4, 8)，默认 dtype=torch.float32
5 memory = x.numel() * 4          # 元素个数 x 4 字节 = 128 字节
```

Listing 1: 创建 FP32 张量并计算内存占用

讲师举例：GPT-3 前馈层中的一个权重矩阵约有 23 GB，可见这些张量可能非常巨大。

1.4.2 FP16：半精度及其局限

FP16（16-bit 浮点数）采用 1 个符号位、5 个指数位和 10 个尾数位。虽然内存占用减半，但其动态范围非常有限——指数仅 5 位，无法准确表示非常大或非常小的数。例如，即使想构建一个数值为 10^8 的张量，FP16 也可能将其直接下溢为零（**underflow**）。

因此，若直接用 FP16 训练模型，会导致训练不稳定，出现下溢、上溢以及 NaN 等问题。

1.4.3 BF16: 动态范围与精度的折中

BF16 (Brain Float 16) 于 2018 年由 Google Brain 提出, 旨在解决 FP16 的动态范围不足问题。它与 FP16 同为 16 位, 但将部分尾数位挪给了指数位——保持 8 位指数 (与 FP32 相同) 和 7 位尾数。

- 动态范围: 与 FP32 相当, 避免了溢出和下溢;
- 精度: 略低于 FP16, 但在深度学习训练中, 这种取舍通常是值得的;
- 训练稳定性: 由于动态范围足够大, BF16 训练比 FP16 稳定得多。

1.4.4 FP8 与 FP4: 极限量化

FP8 于约 4 年前引入并标准化, 实际上有两种变体:

- E4M3: 4 位指数 + 3 位尾数, 侧重动态范围;
- E5M2: 5 位指数 + 2 位尾数, 侧重分辨率。

NVIDIA 的 Transformer Engine 已支持 FP8 训练。更进一步, NVIDIA 还开发了 MVFP4 (Microscaling FP4), 每个值仅 4 比特。乍看之下, 4 位只能表示 -6 到 +6 的极窄范围, 精度极低; 但其实际机制是分块量化 (**block-wise quantization**):

- 张量被划分为若干块 (block);
- 每块内的 4 位数值可根据该块的缩放因子上下缩放;
- 单个数值因此能获得超过 4 位的有效动态范围, 但同一块内的数值共享缩放因子, 无法让相邻值处于完全不同的数量级。

2025 年, NVIDIA 发布了 Nemotron 系列模型 (讲师提及 “Nemotron... Super”, 具体型号以 NVIDIA 官方发布为准²), 即采用 FP4 训练。需要强调的是, 这些极低精度的操作大多由 NVIDIA 的软件栈在底层自动处理, 并非用户直接创建 FP4 张量即可使用。

关于单比特量化的讨论

有同学提问: 是否有人在推动模型往单比特 (1-bit) 方向发展? 讲师回答: 训练和推理是不同的。很多低比特工作属于量化 (**quantization**) ——先用 BF16 训练模型, 再量化到 1-bit 或 2-bit 用于推理。这比直接训练 1-bit 语言模型容易得多, 目前尚未有人成功训练出可用的 1-bit 模型。

1.5 混合精度训练

既然 FP32 占用大量内存, 而 FP16 训练风险太高, BF16 成为了一种折中方案。但即便如此, 现代训练仍普遍采用混合精度训练 (**Mixed Precision Training**):

- **BF16** 用于参数、激活值和梯度;
- **FP32** 用于优化器状态 (如 Adam 的一阶矩和二阶矩)。

²NVIDIA Nemotron 模型官方页面: <https://developer.nvidia.com/nemotron>

使用 FP32 存储优化器状态的原因在于稳定性：优化器需要对梯度进行平方、累加、求平均等操作，BF16 的精度不足可能导致数值不稳定。

在 PyTorch 中，启用混合精度训练非常简单，只需使用 `torch.amp` 库：

```
1 import torch
2 from torch.cuda.amp import autocast, GradScaler
3
4 scaler = GradScaler()
5
6 with autocast(device_type='cuda', dtype=torch.bfloat16):
7     outputs = model(inputs)
8     loss = criterion(outputs, targets)
9
10 scaler.scale(loss).backward()
11 scaler.step(optimizer)
12 scaler.update()
```

Listing 2: PyTorch 混合精度训练示例

`torch.amp` 会自动判断哪些操作可以安全地转为 BF16（如矩阵乘法），哪些操作需要保留 FP32（如指数运算），无需手动干预。

1.6 GPU 内存搬运

张量的内存计算很简单：

$$\text{内存占用} = \text{元素个数} \times \text{每个元素的字节数} \quad (1)$$

默认情况下，PyTorch 创建的张量位于 CPU 内存上：

```
1 import torch
2
3 x = torch.randn(4, 8)           # 默认在 CPU 上
4 x_gpu = x.cuda()               # 搬到 GPU
5 x_gpu = x.to('cuda')           # 等价写法
```

Listing 3: 张量在 CPU 与 GPU 之间的搬运

若想获得加速，必须将张量显式移动到 GPU 上。讲师特别指出，其演示环境为笔记本 CPU，因此部分代码仅作展示；实际训练应在 GPU 上执行。

本章小结

本章从课程回顾出发，明确了资源核算的核心目标：在有限算力与内存下最大化训练效率。我们介绍了张量作为存储基本单元的地位，系统梳理了 FP32/FP16/BF16/FP8/FP4 等精度类型的特点与权衡，阐述了混合精度训练的现代实践，并强调了 GPU 内存搬运的基础操作。这些知识为后续章节深入讨论计算成本（第 3 章）、算术强度（第 4 章）和训练全流程（第 5 章）奠定了必要的底层基础。

2 einops: 命名维度与爱因斯坦求和

在讨论张量运算的计算成本之前，我们先引入一个强大的张量操作库——`einops`。讲师指出，传统基于索引的张量操作（如 `torch.matmul`、`transpose`、`sum`）容易出

错且可读性差，而 `einops` 以命名维度（**named dimensions**）取代隐式索引，使转置、求和、批处理、维度拆分与合并变得直观且模块化。本章介绍 `einops` 的三大核心操作：`einsum`（爱因斯坦求和）、`reduce`（广义规约）与 `rearrange`（维度重排），它们不仅是后续 Transformer 实现的重要工具，也为理解反向传播中的梯度计算提供了清晰的符号体系。

2.1 设计动机：索引 vs. 命名维度

考虑一段传统的 PyTorch 矩阵乘法代码：

```
1 import torch
2
3 X = torch.randn(3, 4)    # 3x4 矩阵
4 Y = torch.randn(4, 3)    # 4x3 矩阵
5 Z = torch.matmul(X, Y)    # 结果：3x3
```

Listing 4: 传统索引式矩阵乘法

若涉及转置与批量维度，代码会迅速变得晦涩：

```
1 # X: (2, 3, 4), Y: (2, 4, 3)
2 Z = torch.matmul(X, Y.transpose(1, 2)) # transpose(1, 2) 的含义需靠推断
```

Listing 5: 传统方式：批量矩阵乘法

`einops` 的核心思想是：为每个维度赋予语义名称，让操作本身说明维度的对应关系，而非依赖容易混淆的索引数字。上述例子在 `einops` 中可写为：

```
1 from einops import einsum
2
3 # X: (seq1=3, hidden=4), Y: (hidden=4, seq2=3)
4 Z = einsum(X, Y, "seq1 hidden, hidden seq2 -> seq1 seq2")
```

Listing 6: `einops` 命名维度写法

在代码 6 中，`hidden` 维度同时出现在两个输入中，因此会被求和消去；输出仅保留 `seq1` 与 `seq2`。这种写法无需显式转置，也无需记忆维度序号，极大降低了出错概率。

2.2 `einsum`：广义矩阵乘法

`einsum`（爱因斯坦求和，Einstein Summation）可视为广义矩阵乘法。其规则如下：

- 每个输入张量的维度以空格分隔的标识符命名；
- 出现在多个输入中但未出现在输出中的标识符，表示对该维度求和；
- 输出中的标识符顺序决定了结果张量的维度排列。

2.2.1 矩阵乘法

设 $\mathbf{X} \in \mathbb{R}^{M \times K}$ ， $\mathbf{Y} \in \mathbb{R}^{K \times N}$ ，则经典矩阵乘法 $\mathbf{Z} = \mathbf{XY}$ 的 `einsum` 表示为：

```
1 from einops import einsum
2 import torch
3
4 X = torch.randn(3, 4)
5 Y = torch.randn(4, 5)
6
```



```

7 # "m k, k n -> m n": k 为求和维度
8 Z = einsum(X, Y, "m k, k n -> m n") # 结果: (3, 5)

```

Listing 7: einsum 实现矩阵乘法

从数学上看，式 2 等价于：

$$Z_{m,n} = \sum_k X_{m,k} Y_{k,n}. \quad (2)$$

2.2.2 批量矩阵乘法

当输入带有批量维度时，传统 PyTorch 写法需要处理转置和广播，代码晦涩且容易出错。讲师在课上展示了“旧写法”：

```

1 # X: (batch=2, seq1=3, hidden=4)
2 # Y: (batch=2, seq2=3, hidden=4) # 注意 Y 的最后两维与 X 不匹配
3 # 旧写法: 需要手动转置最后两个维度, 并依赖广播机制
4 Z = X @ Y.transpose(-2, -1) # transpose(-2, -1) 的含义需靠推断
5 # 结果: (2, 3, 3)

```

Listing 8: 传统方式: 批量矩阵乘法的晦涩写法

代码 8 中的 `transpose(-2, -1)` 不仅难以阅读，而且当张量秩 (rank) 变化时 (例如增加 head 维度)，索引数字会随之改变，极易引入 bug。einsum 则直接通过命名解决：

```

1 # X: (batch=2, seq1=3, hidden=4)
2 # Y: (batch=2, hidden=4, seq2=3)
3 Z = einsum(X, Y, "batch seq1 hidden, batch hidden seq2 -> batch seq1 seq2")
4 # 结果: (2, 3, 3)

```

Listing 9: einsum 实现批量矩阵乘法

注意代码 9 中无需显式转置：由于 Y 的维度顺序是 hidden seq2，而 X 是 seq1 hidden，命名本身已隐含了正确的乘法方向。

2.2.3 省略号: 批处理维度的泛化

在语言模型中，张量往往带有多个批处理维度 (如 batch、seq、head)。einsum 支持用 ... (省略号) 代表任意数量的前导维度，从而编写与具体形状无关的模块化代码：

```

1 # X: 形状为 (... , seq1, hidden)
2 # Y: 形状为 (... , hidden, seq2)
3 # "..." 自动匹配所有前导批处理维度
4 Z = einsum(X, Y, "... seq1 hidden, ... hidden seq2 -> ... seq1 seq2")

```

Listing 10: 省略号泛化批处理维度

讲师特别强调，这种写法在 Transformer 的多头注意力中极为实用：当张量带有 batch、seq、head 等多个前导维度时，... 让我们无需关心输入的确切秩 (rank)，代码更具复用性。

2.3 reduce: 广义规约操作

reduce 是对 sum、mean、max、min 等规约操作的统一抽象。其语法与 einsum 类似：输入维度中未在输出中出现的标识符将被规约。

```

1 from einops import reduce
2
3 # X: (batch=2, seq=3, hidden=4)
4 # 对 hidden 维度求和, 保留 batch 与 seq
5 Z = reduce(X, "batch seq hidden -> batch seq", "sum")
6 # 结果: (2, 3)

```

Listing 11: reduce 对指定维度求和

代码 11 等价于 `X.sum(dim=-1)`, 但优势在于维度含义一目了然。reduce 支持的规约方式包括 "sum"、"mean"、"max"、"min" 等:

```

1 # 沿 seq 维度取均值
2 Z_mean = reduce(X, "batch seq hidden -> batch hidden", "mean")
3
4 # 沿 hidden 维度取最大值
5 Z_max = reduce(X, "batch seq hidden -> batch seq", "max")

```

Listing 12: reduce 支持多种规约方式

讲师指出, reduce 本质上是与底层操作 (如 `torch.sum`) 相同的计算, 可视为语法糖, 性能并无损失, 但代码可读性大幅提升。

2.4 rearrange: 维度的拆分与合并

`rearrange` 是 `einops` 中最强大的操作之一, 用于拆分 (**split**) 和合并 (**merge**) 维度, 而无需显式的 `reshape` 或 `view`。它遵循与 `einsum` 相同的命名约定, 但支持用括号 (a b) 表示将单一维度拆分为两个维度的乘积。

2.4.1 注意力头示例: 拆分隐藏维度

在多头注意力 (Multi-Head Attention) 中, 一个形状为 (batch, hidden) 的隐藏向量往往被拆分为 (batch, head, hidden_per_head)。假设 `hidden = 8`, 我们希望拆分为 `head=2` 个头, 每头 `hidden_per_head=4`:

```

1 from einops import rearrange
2
3 X = torch.randn(3, 8) # (batch=3, hidden=8)
4
5 # 将 hidden 拆分为 (head hidden_per_head)
6 X_heads = rearrange(X, "batch (head hidden_per_head) -> batch head
7                      hidden_per_head",
8                      head=2)
9 # 结果: (3, 2, 4)

```

Listing 13: rearrange 拆分注意力头维度

代码 13 中, (head hidden_per_head) 表示原维度 `hidden` 是 `head` 与 `hidden_per_head` 的乘积。通过指定 `head=2`, `einops` 自动推断 `hidden_per_head = 8 / 2 = 4`。

拆分后, 可对每个头独立进行线性变换, 再合并回原始形状:

```

1 # W: (head=2, hidden_per_head=4, out=4)
2 W = torch.randn(2, 4, 4)
3
4 # 对每个头做矩阵乘法: (batch, head, hidden_per_head) @ (head,
5                      hidden_per_head, out)

```

```

5 Y = einsum(X_heads, W, "batch head hidden, head hidden out -> batch head
   out")
6 # 结果: (3, 2, 4)
7
8 # 合并回 (batch, hidden)
9 Y_merged = rearrange(Y, "batch head hidden -> batch (head hidden)")
10 # 结果: (3, 8)

```

Listing 14: 拆分-变换-合并的完整流程

2.4.2 行主序约定

当使用 `rearrange` 将多个维度合并为一个维度时（如代码 14 中的 `"batch head hidden -> batch (head hidden)"`），`einops` 默认遵循行主序（**row-major**）约定：最右侧的维度变化最快，最左侧的维度变化最慢。

例如，将二维张量 (a, b) 展平为一维时：

```

1 X = torch.tensor([[1, 2, 3],
2                   [4, 5, 6]]) # (a=2, b=3)
3
4 flat = rearrange(X, "a b -> (a b)")
5 # 结果: tensor([1, 2, 3, 4, 5, 6])
6 # 等价于 X.view(-1)，遵循行主序

```

Listing 15: `rearrange` 的行主序展平

讲师在课上特别回应了学生的提问：将二维合并为一维时，顺序由 `rearrange` 表达式中的括号内顺序决定。在代码 15 中，`(a b)` 表示先遍历 b （行内），再遍历 a （行间），这正是 NumPy 与 PyTorch 默认的行主序（C-order）约定。

2.5 小结

`einops` 通过命名维度统一了张量操作的核心模式，其三大构件的功能可概括如下：

- **einsum**: 广义矩阵乘法，隐式完成转置、广播与求和，支持 ... 泛化任意批处理维度；
- **reduce**: 对指定命名维度进行求和、均值、最大/最小值等规约，语义清晰；
- **rearrange**: 通过括号语法拆分或合并维度，天然支持多头注意力等场景，默认遵循行主序。

如讲师所言，一旦掌握 `einops` 的思维方式，转置、规约与维度操作将变得流畅自然。更重要的是，在第 5 章讨论反向传播时，我们将看到 `einsum` 如何用完全相同的命名规则表达链式法则中的梯度计算，从而避免传统微积分中容易混淆的矩阵转置与乘法顺序问题。

3 计算成本：FLOPs、硬件峰值与 MFU

3.1 FLOPs 与 FLOPS 的辨析

在估算模型训练所需的计算资源时，首先需要厘清两个极易混淆的概念：**FLOPs**（floating-point operations，浮点运算次数）与 **FLOPS**（floating-point operations per second，每秒浮点运算次数）。

- **FLOPs**（小写 s）：衡量的是计算总量，即完成某项任务一共需要执行多少次浮点运算。例如，GPT-3 的训练大约消耗了 3×10^{23} 次浮点运算，这里的单位就是 FLOPs。
- **FLOPS**（大写 S 或写作 flop/s）：衡量的是硬件速度，即加速器每秒钟能够完成的浮点运算次数。例如，NVIDIA H100 GPU 在 BF16 精度下的标称峰值约为 989 TFLOPS。

讲师特别强调，这两个术语在口语中经常被混用，但在进行资源核算时必须严格区分。我们在后续讨论中会始终使用“FLOPs”表示运算总量，“FLOPS”表示硬件吞吐率。

3.2 NVIDIA H100 规格表解读

要估算训练时间，必须先了解所用硬件的理论峰值性能。表 2 列出了 NVIDIA H100 SXM5 GPU 的关键规格参数，数据来源于 NVIDIA 官方规格表³。

Table 2: NVIDIA H100 SXM5 关键规格（BF16 精度）

指标	数值
BF16 Tensor Core 峰值（稀疏模式）	1,979 TFLOPS
BF16 Tensor Core 峰值（稠密模式）	≈ 989 TFLOPS
HBM 容量	80 GB
HBM 内存带宽	3.35 TB/s

重要脚注：稀疏模式。规格表上最醒目的数字通常是 1,979 TFLOPS，但这对应的是稀疏矩阵加速模式（sparse mode）。对于常规的稠密矩阵运算，实际峰值需要将该数字除以 2，即约 989 TFLOPS。讲师指出，初次查阅规格表时很容易被宣传数字误导，跑基准测试后才发发现实际测不到标称值，这时才注意到脚注中的小字说明。在后续的所有计算中，我们均采用稠密模式下的 989 TFLOPS 作为 H100 的标称峰值。

为了建立直观感受，我们可以做一道简单的数量级估算：一块 H100 以峰值运行一周，所能提供的总计算量为

$$989 \times 10^{12} \text{ FLOPS} \times 7 \times 24 \times 3600 \text{ s} \approx 6 \times 10^{20} \text{ FLOPs.} \quad (3)$$

这个量级与大型语言模型训练所需的 10^{23} 次浮点运算相比，让我们对硬件能力与模型规模之间的差距有了初步认识。

3.3 线性层 FLOPs 的推导

深度神经网络中，计算成本的核心来源是矩阵乘法。我们以最常见的线性层（全连接层）为例，逐步推导其前向传播所需的浮点运算次数。

设定：设输入数据矩阵为 $X \in \mathbb{R}^{B \times D_{\text{in}}}$ ，其中 B 为批次大小（batch size）， D_{in} 为输入维度；权重矩阵为 $W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ ， D_{out} 为输出维度。线性层的前向传播为

$$Y = XW, \quad Y \in \mathbb{R}^{B \times D_{\text{out}}}. \quad (4)$$

推导步骤：

³NVIDIA H100 Tensor Core GPU Architecture: <https://www.nvidia.com/en-us/data-center/h100/>

1. 结果矩阵 Y 共有 $B \times D_{\text{out}}$ 个元素。
2. 每个元素 Y_{ij} 是 X 的第 i 行与 W 的第 j 列的点积，涉及 D_{in} 次乘法和 $D_{\text{in}} - 1$ 次加法。
3. 严格来说，每个元素的运算量为 D_{in} 次乘法 + $(D_{\text{in}} - 1)$ 次加法。但对于大规模矩阵，减去的 1 可以忽略不计。
4. 若将一次乘法和一次加法视为一个基本计算单元（共 2 次浮点运算），则总运算量为

$$\text{FLOPs}_{\text{linear}} = 2 \cdot B \cdot D_{\text{in}} \cdot D_{\text{out}}. \quad (5)$$

式 (5) 是资源核算中最常用的公式之一。讲师还提供了另一种直观的理解角度：注意到权重矩阵 W 的参数数量为 $D_{\text{in}} \cdot D_{\text{out}}$ ，因此

$$\text{FLOPs}_{\text{linear}} = 2 \cdot B \cdot (\text{参数量}) = 2 \cdot (\text{数据点数量}) \cdot (\text{参数量}). \quad (6)$$

这一视角将在第 5 章中推广到整个深度网络，导出著名的“ $6ND$ ”训练总 FLOPs 公式。

3.4 逐元素操作的 FLOPs

与矩阵乘法相比，逐元素操作（element-wise operations）的计算量通常小得多。例如，对两个形状相同的张量做逐元素加法：

$$Z_{ij} = X_{ij} + Y_{ij}, \quad (7)$$

每个元素仅需要 1 次浮点运算（加法），因此总 FLOPs 等于张量的元素个数。若张量形状为 $B \times D$ ，则

$$\text{FLOPs}_{\text{element-wise}} = B \cdot D. \quad (8)$$

讲师指出，对于足够大的矩阵，矩阵乘法是最耗时的操作，因此在粗略估算模型总计算量时，通常只统计矩阵乘法的 FLOPs，而忽略逐元素操作（如激活函数、偏置加法等）。但这里有一个重要前提：这一简化仅在计算受限（compute-bound）的场景下成立。如果操作受限于内存带宽，即使是看似廉价的逐元素操作也可能成为瓶颈——这正是第 4 章将要深入讨论的主题。

3.5 GPU 计时方法与 CUDA 同步

FLOPs 只回答了“需要多少计算”的问题，要回答“实际需要多久”，必须在真实硬件上进行计时。

3.5.1 为什么需要 `torch.cuda.synchronize()`

GPU 的执行是异步的：当你调用一个 PyTorch 算子时，CPU 会立即将内核（kernel）提交给 GPU 的队列并返回，而不会等待 GPU 实际完成计算。如果直接在算子调用前后用 Python 的 `time.time()` 计时，测得的往往是 CPU 提交时间而非 GPU 实际执行时间，结果会异常地快。

正确的计时方法如代码 16 所示：

```

1 import torch
2 import time
3
4 # 预热：避免首次启动开销
5 for _ in range(10):
6     output = model(input)
7
8 torch.cuda.synchronize() # 确保之前所有 GPU 操作已完成
9 start = time.time()
10
11 for _ in range(num_runs):
12     output = model(input)
13
14 torch.cuda.synchronize() # 等待本次操作真正完成
15 end = time.time()
16
17 elapsed = (end - start) / num_runs # 单次运行的平均时间

```

Listing 16: GPU 计时标准模板

3.5.2 多次运行取平均

由于 GPU 计时存在噪声（如操作系统调度、温度节流、内存分配开销等），通常建议执行多次运行并取平均值。讲师推荐的实践是：先进行若干次预热（warm-up）迭代以排除首次启动开销，再进行正式计时。

3.5.3 从计时结果计算实际 FLOPS

测得实际运行时间 T 后，即可计算实际达到的 **FLOPS**：

$$\text{Actual FLOPS} = \frac{\text{Total FLOPs}}{T}. \quad (9)$$

3.6 MFU：模型浮点利用率

3.6.1 定义

模型浮点利用率（Model FLOPs Utilization, MFU）是衡量模型在硬件上运行效率的核心指标，定义为实际达到的 FLOPS 与硬件标称峰值 FLOPS 的比值：

$$\text{MFU} = \frac{\text{Actual FLOPS}}{\text{Peak FLOPS}} = \frac{\text{Total FLOPs}/T}{\text{Peak FLOPS}}. \quad (10)$$

式 (10) 中的“Peak FLOPS”应采用稠密模式下的标称值（H100 BF16 为 989 TFLOPS）。讲师特别说明，MFU 的计算暂不考虑通信和其他系统开销，它纯粹反映计算内核本身对硬件峰值算力的利用程度。

3.6.2 典型值与工程含义

根据讲师的经验，MFU 的典型取值范围如下：

- **MFU ≈ 0.5** ：现代大规模模型训练中的常见值，属于不错的表现。考虑到模型中除了矩阵乘法外，还包含激活函数、归一化、注意力等开销，能达到峰值的一半已属高效。

- **MFU ≈ 0.8** : 仅在纯粹的矩阵乘法基准测试中才能达到，实际模型训练几乎不可能达到这一数值。
- **MFU ≈ 0.1** : 说明存在严重问题，可能是批大小过小、数据加载瓶颈、不必要的同步、或算子实现低效等原因，需要立即排查优化。

3.6.3 为什么 MFU 通常只有 0.5?

在讲座的 39:19 处，有学生提问：“为什么实际模型利用率只有 50%?” 讲师回答：“等会儿讲到内存瓶颈时我再回答这个问题。”这一悬念将在第 4 章揭晓——MFU 偏低的根本原因在于，模型中大量操作（如激活函数、逐元素运算、矩阵-向量乘法）的算术强度（Arithmetic Intensity）远低于加速器的阈值，导致 GPU 大部分时间都在等待数据从内存搬运到计算核心，而非执行实际计算。矩阵乘法之所以能成为 Transformer 设计的核心，正是因为它具有足够高的算术强度，能够让加速器接近满负荷运转。

3.7 本章小结

本章从 FLOPs 与 FLOPS 的概念辨析出发，建立了计算成本核算的基本框架：

1. **FLOPs** 衡量计算总量，**FLOPS** 衡量硬件速度，两者不可混淆。
2. 线性层前向传播的 FLOPs 为 $2 \cdot B \cdot D_{\text{in}} \cdot D_{\text{out}}$ ，这是深度网络计算估算的基石。
3. 逐元素操作的 FLOPs 等于元素个数，通常远小于矩阵乘法。
4. GPU 计时时必须使用 `torch.cuda.synchronize()` 消除异步执行带来的测量误差，并多次取平均。
5. **MFU** = 实际 FLOPS / 标称峰值 FLOPS，现代模型约 0.5 为良好，0.1 则意味着存在严重瓶颈。

在下一章（第 4 章）中，我们将引入算术强度和屋顶线模型（Roofline Model），从理论上解释为什么 MFU 难以接近 1.0，以及 Transformer 的架构设计如何巧妙地避开了内存带宽瓶颈。

4 算术强度与屋顶线模型

在第 3 章中，我们讨论了如何估算模型的浮点运算次数（FLOPs）以及如何利用 MFU（Model FLOPs Utilization, MFU）衡量实际运行效率。然而，一个自然的问题是：为什么现代模型的 MFU 通常只有 0.5 左右？为什么纯矩阵乘法可以达到 0.8，而完整模型的利用率却明显更低？要回答这个问题，我们需要引入算术强度（Arithmetic Intensity）和屋顶线模型（Roofline Model）这两个核心概念。本章将说明：内存带宽瓶颈——而非单纯的计算能力不足——才是限制 MFU 的关键因素。

4.1 硬件简图：计算与通信的双重瓶颈

4.1.1 HBM 与计算核心

现代 GPU 加速器（如 NVIDIA H100）的硬件结构可以简化为两个核心部件：高带宽内存（High Bandwidth Memory, HBM）和计算核心（Compute Core）。如图 1 所示，所有待

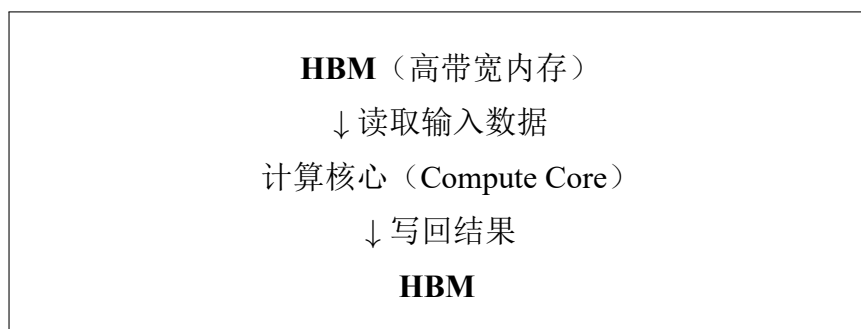


Figure 1: GPU 硬件简图：数据在 HBM 与计算核心之间流动。

处理的张量数据首先存储在 HBM 中；在执行计算之前，这些数据需要被搬运到计算核心附近的缓存或寄存器中；计算完成后，结果再被写回 HBM。

因此，完成一项操作的总时间由两部分构成：

- 计算时间 T_{compute} ：执行浮点运算所需的时间，取决于计算核心的峰值算力（Peak FLOPS）；
- 通信时间 T_{memory} ：将数据从 HBM 搬运到计算核心、并将结果写回所需的时间，取决于内存带宽（Memory Bandwidth）。

4.1.2 计算与通信的重叠

现代 GPU 架构支持一种重要的优化机制：通信与计算重叠（Overlapping Compute and Memory）。具体而言，GPU 无需等待全部数据加载完毕才开始计算；相反，数据可以以流式方式分批到达，计算核心一旦获得足够的数据便立即开始运算，同时后续数据仍在持续传输。这种流水线式的执行模式意味着，在理想情况下，总时间并非两者简单相加，而是取两者中的较大值：

$$T = \max(T_{\text{compute}}, T_{\text{memory}}). \quad (11)$$

式 (11) 是本节分析的基础。当然，实际硬件中无法做到完美的重叠，总会存在一定的额外开销；但对于数量级的估算和瓶颈判断而言，这一假设已经足够准确。

4.2 算术强度的定义

4.2.1 操作强度与加速器强度

为了量化一项操作是“计算受限”还是“内存受限”，我们引入算术强度（Arithmetic Intensity），记为 I ，其定义为每传输一字节数据所执行的浮点运算次数：

$$I = \frac{\text{FLOPs}}{\text{Bytes}}. \quad (12)$$

类似地，我们可以定义加速器强度阈值（Accelerator Intensity Threshold），记为 I_{accel} ，它表示硬件每传输一字节最多能执行多少浮点运算：

$$I_{\text{accel}} = \frac{\text{Peak FLOPS}}{\text{Bandwidth}}. \quad (13)$$

以 NVIDIA H100 为例（BF16 稠密模式），其标称峰值算力约为 989 TFLOPS，内存带宽约为 3.35 TB/s，因此：

$$I_{\text{accel}}^{(\text{H100})} = \frac{989 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOP/Byte} \approx 300 \text{ FLOP/Byte}. \quad (14)$$

这是一个非常好记的数字：约 300。它构成了判断瓶颈的分水岭。

4.2.2 内存受限与计算受限的判别

将式 (12) 与式 (13) 结合，我们可以得到如下判别准则：

- 若 $I < I_{\text{accel}}$ （即操作的算术强度低于加速器阈值），则 $T_{\text{memory}} > T_{\text{compute}}$ ，操作处于内存受限（Memory-bound）状态——大部分时间花在等待数据上；
- 若 $I > I_{\text{accel}}$ （即操作的算术强度高于加速器阈值），则 $T_{\text{compute}} > T_{\text{memory}}$ ，操作处于计算受限（Compute-bound）状态——计算核心满负荷运转。

这一判别准则与式 (11) 完全等价，只是从“时间比”转换为了“强度比”，更便于对不同操作进行横向比较。

4.3 典型操作的算术强度对比

下面我们对几种深度学习中最常见的操作逐一计算其算术强度，并判断其瓶颈类型。为统一口径，以下计算均假设数据类型为 BF16（每个元素占 2 字节）。

4.3.1 ReLU 激活函数： $I \approx 0.25$

考虑一个长度为 N 的 BF16 向量 $\mathbf{x}$ ，对其逐元素应用 ReLU 激活函数： $y_i = \max(x_i, 0)$ 。

- 数据搬运量：读取 $\mathbf{x}$ 需 $2N$ 字节，写回 $\mathbf{y}$ 需 $2N$ 字节，总计 $4N$ 字节；
- 浮点运算次数：每个元素执行一次与零的比较（计为 1 FLOP），总计 N FLOPs。

因此，ReLU 的算术强度为：

$$I_{\text{ReLU}} = \frac{N}{4N} = 0.25 \text{ FLOP/Byte.} \quad (15)$$

由于 $0.25 \ll 300$ ，ReLU 是典型的内存受限操作。计算核心几乎在瞬间即可完成比较，但大量时间消耗在数据的读写上。

4.3.2 GELU 激活函数： $I \approx 5$

GELU（Gaussian Error Linear Unit）是另一种常用的激活函数，其数学形式比 ReLU 复杂得多，涉及指数、平方根等运算，每个元素大约需要 20 次浮点运算。然而，其数据搬运量与 ReLU 完全相同：读取输入 $2N$ 字节，写回输出 $2N$ 字节，共 $4N$ 字节。

因此，GELU 的算术强度约为：

$$I_{\text{GELU}} \approx \frac{20N}{4N} = 5 \text{ FLOP/Byte.} \quad (16)$$

4

尽管 GELU 的计算量是 ReLU 的约 20 倍，但其算术强度 5 仍然远低于 H100 的阈值 300。这意味着：从架构设计的角度看，GELU 与 ReLU 一样，都是内存受限的。两者的实际耗时几乎相同，因为瓶颈根本不在计算本身，而在数据搬运。这是一个非常重要的直觉：不能仅凭操作的数学复杂度判断其运行开销，必须结合内存带宽一起分析。

⁴此处“约 20 FLOPs”为粗略估算，实际 GELU 的硬件实现可能因近似算法（如 tanh 近似或多项式拟合）而有所不同。

4.3.3 点积 (Dot Product): $I \approx 0.5$

考虑两个长度为 N 的 BF16 向量 $\mathbf{x}$ 和 $\mathbf{w}$ 的点积: $y = \sum_{i=1}^N x_i w_i$ 。

- 数据搬运量: 读取 $\mathbf{x}$ 需 $2N$ 字节, 读取 $\mathbf{w}$ 需 $2N$ 字节, 写回标量结果 y 仅需 2 字节 (可忽略), 总计约 $4N$ 字节;
- 浮点运算次数: N 次乘法和 $N - 1$ 次加法, 总计约 $2N$ FLOPs。

因此, 点积的算术强度为:

$$I_{\text{dot}} \approx \frac{2N}{4N} = 0.5 \text{ FLOP/Byte.} \quad (17)$$

0.5 仍然远小于 300, 点积同样是典型的内存受限操作。

4.3.4 矩阵-向量乘法 (Mat-Vec): 内存受限

考虑矩阵 $\mathbf{W} \in \mathbb{R}^{N \times N}$ 与向量 $\mathbf{x} \in \mathbb{R}^N$ 的乘法: $\mathbf{y} = \mathbf{W}\mathbf{x}$, 其中 $\mathbf{y} \in \mathbb{R}^N$ 。

- 数据搬运量: 读取 $\mathbf{x}$ 需 $2N$ 字节, 读取 $\mathbf{W}$ 需 $2N^2$ 字节, 写回 $\mathbf{y}$ 需 $2N$ 字节, 总计约 $2N^2 + 4N$ 字节;
- 浮点运算次数: 相当于 N 个点积, 总计约 $2N^2$ FLOPs。

当 N 较大时, 算术强度近似为:

$$I_{\text{mat-vec}} \approx \frac{2N^2}{2N^2} = 1 \text{ FLOP/Byte.} \quad (18)$$

即便考虑到低阶项, $I_{\text{mat-vec}}$ 的数量级仍在 $O(1)$, 远低于 300。因此, 矩阵-向量乘法同样是内存受限的。

4.3.5 矩阵-矩阵乘法 (Mat-Mat): $I \approx N/3$, 计算受限

最后, 我们来看矩阵-矩阵乘法——这是深度学习中最核心的操作。考虑两个 $N \times N$ 的 BF16 矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 相乘, 得到 $\mathbf{C} = \mathbf{A}\mathbf{B}$ 。

- 数据搬运量: 读取 $\mathbf{A}$ 需 $2N^2$ 字节, 读取 $\mathbf{B}$ 需 $2N^2$ 字节, 写回 $\mathbf{C}$ 需 $2N^2$ 字节, 总计 $6N^2$ 字节;
- 浮点运算次数: 根据第 3 章的推导, $N \times N$ 矩阵乘法需要 $2N^3$ FLOPs。

因此, 矩阵-矩阵乘法的算术强度为:

$$I_{\text{mat-mat}} = \frac{2N^3}{6N^2} = \frac{N}{3} \text{ FLOP/Byte.} \quad (19)$$

这是本章最关键的公式之一。它表明: 矩阵乘法的算术强度与矩阵维度 N 成正比。当 N 足够大时 (例如 $N \geq 900$, 此时 $N/3 \geq 300$), 矩阵乘法将从内存受限转变为计算受限, 从而充分利用 GPU 的峰值算力。

这也解释了为什么深度学习实践中总是强调“大的批量大小 (batch size)”或“大的矩阵尺寸”——只有矩阵足够大, 算术强度才能跨过加速器阈值, 让昂贵的计算核心真正满负荷运转。如果矩阵过小, 即便代码逻辑正确, 运行速度也不会显著提升, 因为瓶颈始终卡在内存带宽上。

4.4 Transformer 的算术强度：训练 vs 推理

4.4.1 训练阶段：矩阵乘为主，计算受限

Transformer 架构的设计在算术强度方面是一个好消息。如讲师所指出的，Transformer 本质上是大规模的矩阵乘法运算，中间穿插了一些逐元素操作（如激活函数、LayerNorm 等）。在训练阶段，由于需要处理整个序列的批量数据，前向传播和反向传播中的核心操作都是矩阵-矩阵乘法（如注意力机制中的 $\mathbf{QK}^\top$ 和 $\mathbf{AV}$ ，以及前馈网络中的线性变换）。只要批量大小和隐藏维度足够大，这些矩阵乘法的算术强度就能超过加速器阈值，使训练处于计算受限状态。这是 Transformer 架构被如此广泛采用的重要原因之一——它在硬件效率方面是经过深思熟虑的。

4.4.2 推理阶段：矩阵-向量为主，内存受限

然而，在推理阶段（尤其是自回归生成），情况发生了根本性变化。推理时，模型一次只生成一个 token，这意味着每一步的核心操作不再是矩阵-矩阵乘法，而是矩阵-向量乘法：用一个向量（当前 token 的表示）去乘以各种权重矩阵。正如我们在上一节中看到的，矩阵-向量乘法的算术强度约为 $O(1)$ ，远低于加速器阈值。因此，Transformer 推理本质上是内存受限的。

这一训练与推理之间的瓶颈差异，是现代大模型系统优化的核心挑战之一。后续课程中将专门讨论推理优化技术（如 KV Cache、量化、speculative decoding 等），其目标正是在内存受限的约束下尽可能提升推理吞吐量。

4.5 屋顶线模型（Roofline Model）

4.5.1 模型描述

屋顶线模型是一种直观可视化算术强度与硬件性能关系的工具。如图 2 所示，该图以算术强度 I (FLOP/Byte) 为横轴，以实际达到的 FLOPS 为纵轴。

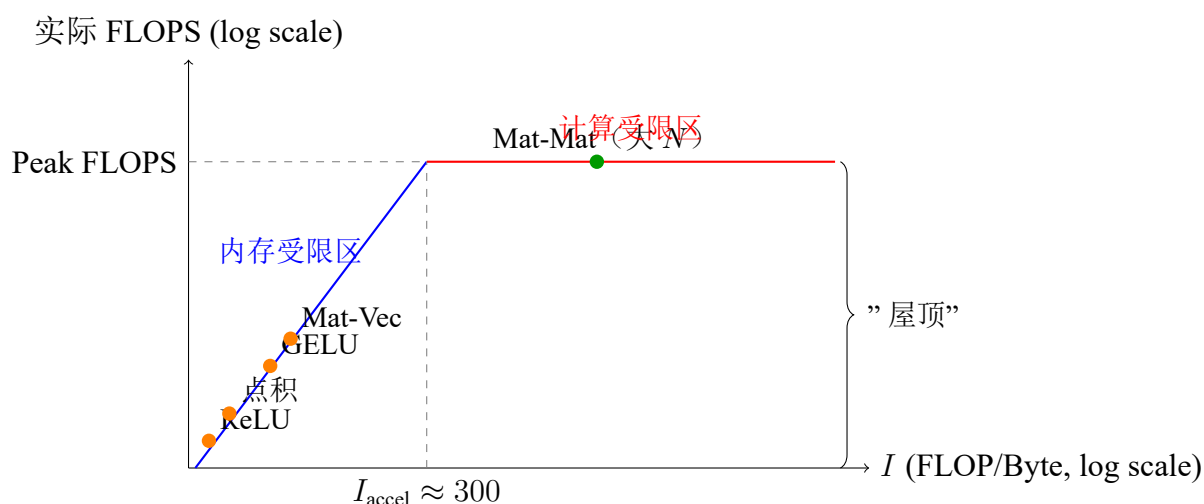


Figure 2: Roofline Plot 示意图 (通常采用对数-对数坐标): 横轴为算术强度 I (FLOP/Byte), 纵轴为实际达到的 FLOPS。左侧斜线区域为内存受限区, 右侧水平区域为计算受限区, 拐点处 $I = I_{\text{accel}} \approx 300$ 。

图中呈现出一个类似“屋顶”的形状:

- 左侧斜线（内存受限区）：当 $I < I_{\text{accel}}$ 时，实际 FLOPS 随算术强度线性增长，斜率等于内存带宽。此时，无论算力多高，性能都被带宽“封顶”。
- 右侧水平线（计算受限区）：当 $I \geq I_{\text{accel}}$ 时，实际 FLOPS 达到并维持在峰值算力附近，不再随 I 增加而提升。
- 拐点：位于 $I = I_{\text{accel}} \approx 300$ 处，是内存受限与计算受限的分界点。

4.5.2 各操作在 Roofline 上的位置

将前面计算的操作标注在 Roofline Plot 上，可以得到清晰的直观认识：

- **ReLU** ($I \approx 0.25$) 和点积 ($I \approx 0.5$) 落在最左侧，实际 FLOPS 仅为峰值的千分之一左右；
- **GELU** ($I \approx 5$) 和矩阵-向量乘法 ($I \approx 1$) 稍靠右，但仍远未达到拐点，实际 FLOPS 仍被带宽严重限制；
- 矩阵-矩阵乘法 ($I \approx N/3$) 当 N 足够大时落在右侧水平区，此时计算核心满负荷运转，实际 FLOPS 接近峰值。

4.6 MFU 偏低与内存瓶颈的关系

现在我们可以回到第 3 章末尾提出的问题：为什么 MFU 通常只有 0.5 左右？

根本原因在于：真实模型并非由纯矩阵乘法构成。Transformer 的前向传播和反向传播中，除了大量的矩阵乘法外，还包含激活函数（ReLU/GELU）、Softmax、LayerNorm、Dropout 等逐元素或低强度操作。这些操作的算术强度远低于加速器阈值，导致 GPU 计算核心在等待数据时空转。即便矩阵乘法部分可以达到 0.8 的利用率，整体 MFU 也会被这些内存受限操作拉低。

因此，MFU 偏低不一定是代码存在 bug，更可能是内存带宽瓶颈的必然结果。理解这一点对于系统优化至关重要：

- 如果 MFU 约为 0.5，这属于现代模型的正常水平；
- 如果 MFU 低至 0.1，则可能确实存在严重的实现问题（如频繁的 CPU-GPU 同步、低效的内存访问模式等），需要排查；
- 如果追求极致的 MFU（如 0.8 以上），则需要让模型尽可能地由大规模矩阵乘法主导——这也是为什么在训练大模型时，工程师会努力增大批量大小和序列长度。

讲师点评

讲师在此提出了一个引人深思的问题：“为什么加速器在空转或等待内存时，还需要如此大的规模？我们能不能设计出性能更优的加速器呢？”这涉及 GPU 微架构的深层设计权衡（如计算核心与 HBM 的面积/功耗配比）。如果你对此有答案，不妨告诉黄仁勋——也许下一代 GPU 就会因你而变得更高效。

4.7 本章小结

本章围绕算术强度和屋顶线模型，系统分析了深度学习操作的性能瓶颈：

1. 现代 GPU 的执行时间由计算时间和通信时间共同决定，理想情况下 $T = \max(T_{\text{compute}}, T_{\text{memory}})$ ；
2. 算术强度 $I = \text{FLOPs}/\text{Bytes}$ 是判断瓶颈类型的关键指标；H100 的加速器阈值约为 300 FLOP/Byte；
3. ReLU ($I \approx 0.25$)、GELU ($I \approx 5$)、点积 ($I \approx 0.5$) 和矩阵-向量乘法均为内存受限操作；
4. 矩阵-矩阵乘法的算术强度为 $I \approx N/3$ ，当矩阵足够大时进入计算受限区，这是 Transformer 训练效率高的根本原因；
5. Transformer 推理以矩阵-向量乘法为主，因此是内存受限的，与训练阶段的瓶颈特性截然不同；
6. Roofline Plot 直观展示了算术强度与实际性能的关系，解释了为什么 MFU 通常只有 0.5 左右——大量低强度操作拖累了整体利用率。

理解算术强度不仅有助于估算模型运行时间，更能指导我们在设计网络架构和编写训练代码时，始终将“每字节搬运能做多少计算”这一核心问题放在心中。

5 训练全流程：前向、反向、优化器与内存占用

5.1 深度网络前向传播

在讨论训练所需的存储与计算之前，我们先建立一个具体的深度网络模型。考虑一个由 L 层全连接层组成的深度网络，输入为批次大小 B 、维度为 D 的数据矩阵 $X \in \mathbb{R}^{B \times D}$ 。每一层包含一个线性变换（权重矩阵 $W_i \in \mathbb{R}^{D \times D}$ ）和一个逐元素激活函数（如 ReLU）。前向传播过程可形式化描述为：

$$H_0 = X, \quad (20)$$

$$Z_i = H_{i-1}W_i, \quad i = 1, \dots, L, \quad (21)$$

$$H_i = \text{ReLU}(Z_i), \quad i = 1, \dots, L. \quad (22)$$

该网络的总参数量为

$$N = L \cdot D^2. \quad (23)$$

代码 17 给出了对应的 PyTorch 实现。

```
1 import torch
2 import torch.nn as nn
3
4 class DeepNet(nn.Module):
5     def __init__(self, depth: int, dim: int):
6         super().__init__()
7         self.layers = nn.ModuleList([
8             nn.Linear(dim, dim) for _ in range(depth)
9         ])
10
11     def forward(self, x: torch.Tensor) -> torch.Tensor:
12         # x: (batch, dim)
```

```

13         for layer in self.layers:
14             x = torch.relu(layer(x))
15         return x

```

Listing 17: 深度学习前向传播示例

讲师指出，Transformer 本质上也是大规模矩阵乘法的堆叠，因此本节以全连接层为例推导出的结论，对 Transformer 同样具有良好的近似性（只要序列长度不至于过长，使得注意力机制的二次复杂度成为主导因素）。

5.2 从标量到矩阵：反向传播的直觉

在正式讨论矩阵形式的反向传播之前，讲师先用一个极简的标量/向量例子建立了直觉。考虑一个一维线性回归场景：

```

1 import torch
2
3 x = torch.tensor([1., 2., 3.])
4 w = torch.tensor([1., 1., 1.], requires_grad=True)
5 target = 10.0
6
7 # 前向：计算预测值与损失
8 pred = x @ w          # 1*1 + 2*1 + 3*1 = 6
9 loss = (pred - target) ** 2 # (6 - 10)^2 = 16
10
11 # 反向：自动计算梯度
12 loss.backward()
13 print(w.grad)         # tensor([-8., -16., -24.])

```

Listing 18: 标量反向传播直觉示例

让我们手动验证 `w.grad` 的结果。设 $p = \mathbf{x} \cdot \mathbf{w} = \sum_i x_i w_i$ ， $\mathcal{L} = (p - t)^2$ 。根据链式法则：

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial w_i} = 2(p - t) \cdot x_i. \quad (24)$$

代入数值： $2(6 - 10) \cdot x_i = -8 \cdot x_i$ ，因此 $\mathbf{w}$ 的梯度为 $[-8, -16, -24]$ ，与 PyTorch 的自动求导结果完全一致。

讲师强调，这个简单例子揭示了一个普遍模式：梯度总是通过链式法则逐层回传——先计算损失对输出的梯度，再乘以输出对参数的局部梯度。一旦理解了这个标量直觉，将其推广到矩阵情形只是维度的扩展，核心逻辑完全相同。

5.3 反向传播与链式法则的 einops 表示

现在我们将上述直觉推广到矩阵情形。以两层简化网络为例：

$$H_1 = XW_1, \quad (25)$$

$$H_2 = H_1W_2, \quad (26)$$

$$\mathcal{L} = \text{loss}(H_2, y). \quad (27)$$

反向传播时，计算图上的每个变量都需要计算其梯度。根据链式法则，对于第 2 层，我们需要计算两类梯度：

1. 对输入的梯度（梯度信号回传）： $\frac{\partial \mathcal{L}}{\partial H_1}$ ；

2. 对参数的梯度（用于参数更新）： $\frac{\partial \mathcal{L}}{\partial W_2}$ 。

利用 **einops** 的命名维度表示，这两个梯度可以写成极为清晰的爱因斯坦求和形式。设批次维度为 `batch`，输入维度为 `in`，输出维度为 `out`：

$$\begin{aligned} \text{grad_}H_1[\text{batch}, \text{in}] &= \text{einsum}(\text{grad_}H_2[\text{batch}, \text{out}], W_2[\text{in}, \text{out}]) \\ &\rightarrow \text{batch in}, \end{aligned} \quad (28)$$

$$\begin{aligned} \text{grad_}W_2[\text{in}, \text{out}] &= \text{einsum}(\text{grad_}H_2[\text{batch}, \text{out}], H_1[\text{batch}, \text{in}]) \\ &\rightarrow \text{in out}. \end{aligned} \quad (29)$$

式 (28) 对 `out` 维度求和，得到回传到上一层的梯度；式 (29) 对 `batch` 维度求和，得到参数梯度。讲师特别强调，这种命名维度的写法避免了传统索引操作中频繁出现的转置困惑——“通过命名就完成了转置”。

5.4 梯度计算量 = 2 × 前向

现在我们来核算反向传播的浮点运算次数。对于单一线性变换 $H_{\text{out}} = H_{\text{in}}W$ ，前向传播的计算量为

$$\text{FLOPs}_{\text{fwd}} = 2 \cdot B \cdot D_{\text{in}} \cdot D_{\text{out}}. \quad (30)$$

反向传播需要执行两次矩阵乘法：

- 计算对输入的梯度：形状为 $(B, D_{\text{out}}) \times (D_{\text{out}}, D_{\text{in}}) \rightarrow (B, D_{\text{in}})$ ，计算量 $2BD_{\text{out}}D_{\text{in}}$ ；
- 计算对权重的梯度：形状为 $(D_{\text{in}}, B) \times (B, D_{\text{out}}) \rightarrow (D_{\text{in}}, D_{\text{out}})$ ⁵，计算量 $2BD_{\text{in}}D_{\text{out}}$ 。

因此，单层的反向传播计算量为

$$\text{FLOPs}_{\text{bwd}} = 2 \times (2BD_{\text{in}}D_{\text{out}}) = 2 \cdot \text{FLOPs}_{\text{fwd}}. \quad (31)$$

讲师总结道：反向传播的梯度计算量正好是前向传播的两倍。这一结论对任何由矩阵乘法构成的网络层都成立。

5.5 总训练 FLOPs = 6 · N · D 的推导

将前向与反向传播的计算量叠加，即可得到训练一个样本（或一个 **token**）所需的总浮点运算次数。对于深度网络，前向传播每层计算量为 $2BD^2$ ，共 L 层，因此

$$\text{FLOPs}_{\text{fwd, total}} = 2BD^2L = 2BN, \quad (32)$$

其中 $N = LD^2$ 为总参数量（见式 (23)）。反向传播为前向的两倍，故

$$\text{FLOPs}_{\text{bwd, total}} = 2 \times \text{FLOPs}_{\text{fwd, total}} = 4BN. \quad (33)$$

于是，处理一批大小为 B 的数据所需的总计算量为

$$\text{FLOPs}_{\text{total}} = (2 + 4)BN = 6BN. \quad (34)$$

若将训练数据总量记为 D （**token** 数量），则整个训练过程的总浮点运算次数为

$$\boxed{\text{FLOPs}_{\text{train}} = 6 \cdot N \cdot D}. \quad (35)$$

式 (35) 是估算大模型训练时间的核心公式。讲师指出，虽然推导基于全连接网络，但该公式对 **Transformer** 同样是一个很好的近似，前提是上下文长度（序列长度）不至于过长——否则注意力机制的 $O(\text{seq}^2)$ 复杂度会带来额外的浮点运算，而这部分未被 $6ND$ 所涵盖。

⁵等价于 $H_{\text{in}}^T \cdot \text{grad_}H_{\text{out}}$ 。

5.6 优化器状态内存：Adagrad 与 Adam

计算出梯度后，下一步是利用优化器更新参数。本节关注优化器带来的额外内存开销。

5.6.1 Adagrad

Adagrad 优化器在 SGD 的基础上引入了梯度的二阶矩（平方梯度的累积和）。对于每个参数 w ，Adagrad 维护一个累积量：

$$G_t = G_{t-1} + (\nabla_w \mathcal{L})^2. \quad (36)$$

因此，Adagrad 的优化器状态大小与参数量 N 相等。由于数值稳定性考虑，这些状态通常以 FP32 存储，每个参数对应 4 字节。

5.6.2 Adam

Adam 优化器同时维护一阶矩（动量）和二阶矩：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_w \mathcal{L}, \quad (37)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_w \mathcal{L})^2. \quad (38)$$

因此，Adam 的优化器状态大小为 $2N$ ，以 FP32 存储时共占用 $8N$ 字节。

Table 3: 常见优化器状态内存占用（每参数）

优化器	状态数量	每参数字节数（FP32）
SGD（无动量）	0	0
Adagrad	1（二阶矩）	4
Adam	2（一阶矩 + 二阶矩）	8

讲师特别指出，优化器状态通常不会成为计算瓶颈，因为优化器更新的计算量远小于反向传播。然而，优化器状态对能否将模型放入 GPU 内存起着决定性作用——这是显存规划中的关键考量。

5.7 训练内存占用核算

综合以上分析，我们可以对训练时的 GPU 内存占用进行全面核算。以混合精度训练（BF16 计算 + FP32 优化器状态）为例，内存主要由以下四部分构成：

1. 参数（**Parameters**）：以 BF16 存储，占用 $2N$ 字节；
2. 梯度（**Gradients**）：以 BF16 存储，占用 $2N$ 字节；
3. 激活值（**Activations**）：以 BF16 存储，每层激活大小为 $B \cdot D$ ，共 L 层，占用 $2 \cdot B \cdot D \cdot L$ 字节；
4. 优化器状态（**Optimizer States**）：以 FP32 存储，Adagrad 为 $4N$ 字节，Adam 为 $8N$ 字节。

Table 4: 训练内存占用分解（混合精度，Adam 优化器）

组件	大小	精度
参数	$2N$	BF16
梯度	$2N$	BF16
激活值	$2 \cdot B \cdot D \cdot L$	BF16
优化器状态 (Adam)	$8N$	FP32
总计	$12N + 2BDL$	—

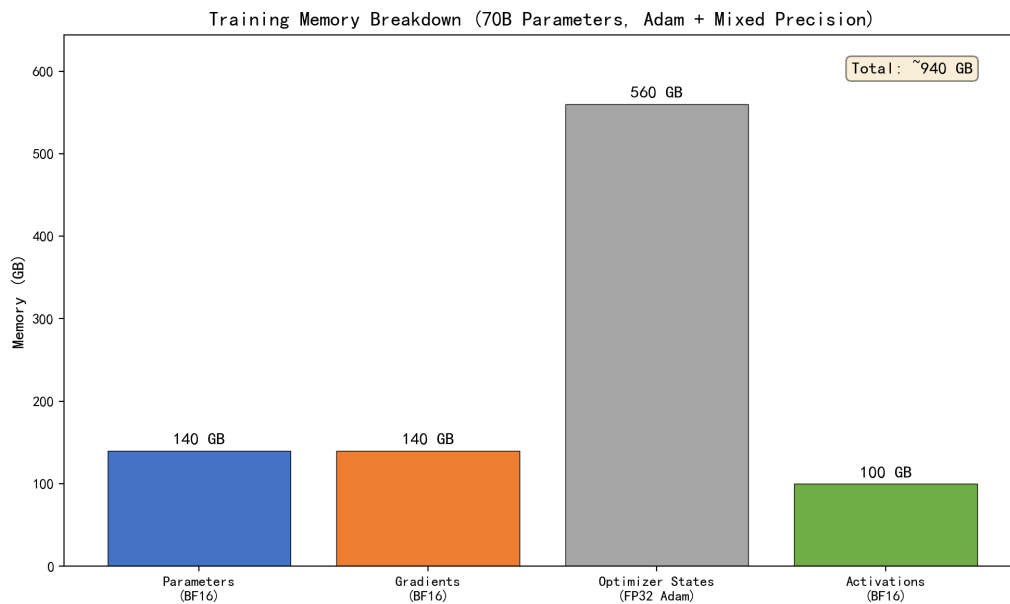
Figure 3: 训练内存占用分解示意图（70B 参数模型，Adam + 混合精度）。参数与梯度各占 $2N$ （BF16），优化器状态占 $8N$ （FP32），激活值大小取决于批次配置。

图 3 以 700 亿参数模型为例，直观展示了各组件的内存占比。可以明显看出，优化器状态（FP32 一阶矩与二阶矩）是单卡训练中最大的内存消耗来源，占总静态内存的约 2/3。这也解释了为什么 8-bit 优化器量化技术（如 bitsandbytes）能显著扩大可训练模型规模——将优化器状态从 8 字节压缩到约 1 字节，静态内存即可从 $12N$ 降至约 $5N$ 。

讲师在课程开头提出的问题——“在 8 块 H100 上能训练的最大模型是多大”——现在可以得到初步回答。假设使用 Adam 优化器、忽略激活值，每块 H100 的 80 GB HBM 可存储的参数量约为

$$N_{\max} \approx \frac{80 \times 10^9}{12} \approx 53 \text{ 亿}. \quad (39)$$

这就是讲师给出的“ $2 + 2 + 8 = 12$ 字节每参数”估算的由来。当然，实际可训练规模还受激活值、通信缓冲区等因素影响，但这一快速估算已能给出数量级的正确判断。

5.8 梯度累积

当批次大小 B 过大导致激活内存超出 GPU 容量时，梯度累积（Gradient Accumulation）是一种标准的内存优化策略。其核心思想是：将大批次拆分为多个微批次（micro-batches），在每个微批次上计算梯度并累加，待累积到目标等效批次大小后再执行参数更新。

```

1 # 假设目标批次大小为 effective_batch_size
2 # 拆分为 num_micro 个微批次
3 accumulation_steps = num_micro
4 optimizer.zero_grad()
5
6 for i, micro_batch in enumerate(dataloader):
7     loss = model(micro_batch) / accumulation_steps
8     loss.backward()           # 梯度自动累加，不清零
9
10    if (i + 1) % accumulation_steps == 0:
11        optimizer.step()      # 更新参数
12        optimizer.zero_grad() # 清零梯度，准备下一轮累积

```

Listing 19: 梯度累积代码示例

梯度累积的优势在于：每个微批次的激活值内存仅与微批次大小成正比，从而大幅降低峰值显存占用。代价是反向传播需要在每个微批次上执行，总计算量不变，但通信与同步开销略有增加。讲师评价这是“一个非常简单的代码改动，就能节省计算资源”。

5.9 激活检查点

5.9.1 原理与内存节省

训练时的另一大内存开销来自激活值。默认情况下，前向传播需要保存每一层的激活值以供反向传播使用，占用 $2 \cdot B \cdot D \cdot L$ 字节。激活检查点（Activation Checkpointing，也称 Gradient Checkpointing）的核心思想是：以计算换内存——在前向传播时只保留部分层的激活值（检查点），反向传播时从上一个检查点重新计算未保存的激活值。

代码 20 展示了在 PyTorch 中使用激活检查点的方式。

```

1 import torch
2 import torch.utils.checkpoint as checkpoint
3
4 class CheckpointedBlock(nn.Module):
5     def __init__(self, dim: int):
6         super().__init__()
7         self.linear = nn.Linear(dim, dim)

```



```

8
9     def forward(self, x: torch.Tensor) -> torch.Tensor:
10         # 只保留输入 x, 不保存中间激活
11         return checkpoint.checkpoint(self._forward, x)
12
13     def _forward(self, x: torch.Tensor) -> torch.Tensor:
14         return torch.relu(self.linear(x))

```

Listing 20: PyTorch 激活检查点使用示例

若对每一层（包含线性层 + 激活函数）都启用检查点，则只需保存每层的输入，而无需保存激活函数前后的全部中间结果。讲师指出，这种做法几乎能省一半内存。其原因在于：在默认训练流程中，每一层反向传播需要两类激活值——激活函数前的线性输出 $Z_i = H_{i-1}W_i$ （用于计算对输入和权重的梯度）以及激活函数后的输出 $H_i = \text{ReLU}(Z_i)$ （用于回传梯度到上一层）。启用检查点后，前向传播时只保留该层的输入 H_{i-1} ；反向传播时，从 H_{i-1} 重新计算 Z_i 和 H_i 即可。因此，存储量从“每层的两个中间张量”降为“每层的一个输入张量”，内存占用几乎减半。

5.9.2 极端情况： L^2 重计算

激活检查点的节省程度取决于检查点的密度。考虑一个极端情况：不保存任何中间激活值，只在网络最前端保留输入。反向传播时，每经过一层都需要从头重新计算所有前置层的激活值。此时，第 i 层的梯度计算需要重算 i 层前向传播，总重计算量为

$$\sum_{i=1}^L i = \frac{L(L+1)}{2} \approx O(L^2). \quad (40)$$

这就是讲师提到的“ L^2 重计算”——在内存上达到了极致效率，但计算开销随层数平方增长，通常不可接受。

5.9.3 最优平衡点： $\sqrt{L}$ 个检查点

在内存与计算之间寻求最优平衡，是一个经典的优化问题。假设我们在网络中均匀设置 K 个检查点，将 L 层分为 K 段，每段约 L/K 层。此时：

- 激活内存：需要保存 K 个检查点输入，每段反向时重算 L/K 层激活，峰值激活内存约为 $O(K \cdot B \cdot D + B \cdot D \cdot L/K)$ ；
- 重计算开销：每段反向需重算 L/K 层前向，总重计算量约为 $O(L \cdot L/K) = O(L^2/K)$ 。

讲师给出的最优策略是：取 $K = \sqrt{L}$ 个检查点。此时

$$\text{激活内存} \propto \sqrt{L}, \quad \text{重计算开销} \propto \sqrt{L}. \quad (41)$$

两者都随 $\sqrt{L}$ 增长，达到了内存与计算开销的帕累托最优平衡。在实际工程中，PyTorch 的 `torch.utils.checkpoint` 默认采用的策略与此思想一致。

5.10 本章小结

本章系统梳理了深度网络训练的完整流程及其资源核算：

- 反向传播通过链式法则计算梯度，利用 einops 的命名维度可清晰表达梯度传播与参数梯度计算；
- 单层反向传播的计算量是前向的两倍，因此总训练 FLOPs 为 $6ND$ （式 (35)），这是估算训练时间的核心公式；
- 优化器状态（尤其是 Adam 的一阶/二阶矩）是显存的重要消耗来源，每参数可达 8 字节（FP32）；
- 梯度累积通过拆分批次降低激活内存，是显存受限时的标准实践；
- 激活检查点以计算换内存，最优策略是在 $\sqrt{L}$ 处设置检查点，使内存与重计算开销均保持为 $O(\sqrt{L})$ 。

这些技术共同构成了现代大模型训练中的内存-计算权衡工具箱。在下一讲中，我们将进入网络架构的具体设计，探讨 Transformer 如何利用矩阵乘法的高算术强度实现高效训练。

总结与展望

本讲从底层到上层系统性地梳理了深度学习训练中的资源核算方法。核心要点可归纳为以下五大支柱：

1. **精度与内存**：FP32 $\rightarrow$ FP16 $\rightarrow$ BF16 $\rightarrow$ FP8 $\rightarrow$ FP4 的演进使大模型训练成为可能；混合精度训练（BF16 计算 + FP32 优化器状态）是现代训练的标准配置。
2. **einops 抽象**：以命名维度取代索引操作，极大提升了张量变换代码的可读性与模块化程度，是后续 Transformer 实现的重要工具。
3. **FLOPs 核算**：线性层 $2BDK$ 、总训练 $6ND$ （前向 2 + 反向 4）是估算训练时间与成本的核心公式。
4. **MFU 与算术强度**：MFU 低不一定是代码 bug，可能是内存带宽瓶颈；算术强度 $I = \text{FLOPs}/\text{Bytes}$ 决定操作是内存受限还是计算受限。
5. **内存优化技术**：梯度累积（以时间换批次大小）和激活检查点（以计算换内存）是显存受限场景下的标准实践。

下一讲将深入网络架构细节，为理解 Transformer 的内部机制奠定基础。